Computer Physics Communications 219 (2017) 121–134

Contents lists available at ScienceDirect

Computer Physics Communications

journal homepage: www.elsevier.com/locate/cpc

Cost efficient CFD simulations: Proper selection of domain partitioning strategies

Bahram Haddadi *, Christian Jordan, Michael Harasek

Technische Universität Wien, Institute of Chemical Engineering, Getreidemarkt 9/1662, A-1060 Vienna, Austria

ARTICLE INFO

Article history:
Received 13 May 2016
Received in revised form 10 February 2017
Accepted 18 May 2017
Available online 29 May 2017

Keywords:
CFD
HPC
Green computing
Parallel efficiency
Domain partitioning
Packed bed adsorption

ABSTRACT

Computational Fluid Dynamics (CFD) is one of the most powerful simulation methods, which is used for temporally and spatially resolved solutions of fluid flow, heat transfer, mass transfer, etc. One of the challenges of Computational Fluid Dynamics is the extreme hardware demand. Nowadays super-computers (e.g. High Performance Computing, HPC) featuring multiple CPU cores are applied for solving—the simulation domain is split into partitions for each core. Some of the different methods for partitioning are investigated in this paper.

As a practical example, a new open source based solver was utilized for simulating packed bed adsorption, a common separation method within the field of thermal process engineering. Adsorption can for example be applied for removal of trace gases from a gas stream or pure gases production like Hydrogen. For comparing the performance of the partitioning methods, a 60 million cell mesh for a packed bed of spherical adsorbents was created; one second of the adsorption process was simulated. Different partitioning methods available in OpenFOAM® (*Scotch*, *Simple*, and *Hierarchical*) have been used with different numbers of sub-domains. The effect of the different methods and number of processor cores on the simulation speedup and also energy consumption were investigated for two different hardware infrastructures (Vienna Scientific Clusters VSC 2 and VSC 3). As a general recommendation an optimum number of cells per processor core was calculated. Optimized simulation speed, lower energy consumption and consequently the cost effects are reported here.

1. Introduction

Modeling fluid dynamic and chemical engineering processes using Computational Fluid Dynamics (CFD) methods [1,2] is getting more and more common. CFD simulations can provide detailed information about flow and heat transfer phenomena happening inside reactors, vessels or equipment where reliable measurements are impossible or very difficult and costly. One of the biggest drawbacks of CFD simulations is the huge ratio of execution time (clock time) to real simulation (physical) time. These days, with getting access to huge computational resources and super-computers, this ratio can be decreased by doing parallel computing. However, important factors of parallel computing using shared or distributed memory hardware are [3].

- Splitting of the computational grid into smaller portions (partitioning or in CFD also known as decomposition) to be assigned to the individual CPU cores.
- Communication between portions (e.g. using Message Passing Interface (MPI) [4]. Open Multi-Processing (OpenMP) [5].
- Collection of the results from individual nodes (reconstruction).

Typically, some methods providing these parallel computing features are commonly implemented in current CFD programs like Ansys FLUENT® [6], CD-Adapco StarCCM++® [7] or OpenFOAM® [8]. All of these codes are based on the finite volume discretization method [9], which solves all relevant equations on a discrete computational grid or mesh. OpenFOAM® is released as open source software (General Public License, GPL) [10] and it is free of license fees. Offering free access to the source code provides a good infrastructure for implementing new models and ideas. Also using Message Passing Interface (MPI) for parallelization makes it a good choice for undertaking simulations with huge number of cells [11].

Miao Wang et al. [12] investigated available partitioning methods to split the generated CFD mesh into several domains and

* Corresponding author.
E-mail addresses: bahram.haddadi.sisakht@tuwien.ac.at (B. Haddadi), christian.jordan@tuwien.ac.at (C. Jordan), michael.harasek@tuwien.ac.at (M. Harasek).

Abbreviations: CFD, Computational Fluid Dynamics; HPC, High Performance Computing; MPI, Message Passing Interface; GPL, GNU Public License.

http://dx.doi.org/10.1016/j.cpc.2017.05.014
0010-4655/

assign them onto different processors in preparation for parallel execution in OpenFOAM® and the partitioning time needed for each method depending on the number of processor patches. They found out *Simple* had the biggest communication overhead while *Scotch* had the biggest partitioning time. Depending on the simulation time the partitioning of meshes might even be larger than simulation time.

Shannon Keough [13] evaluated available open source and commercial compilers and MPI libraries with different compiler flags on EMSMA group Intel based HPC cluster to compare the performance for OpenFOAM® simulations. He claims that open source compiler and MPI libraries and also running jobs using the "bond-to-core" and "bysocket" MPI flags were more efficient.

Chevalier et al. [14] looked at the efficiency of different *Scotch* algorithms and found scalability issues. Andras Horvath et al. [15] found out by increasing the number of cores from one to eight, parallel efficiency will decrease but the overall speedup will be still quite reasonable. Harasek et al. [16] also investigated parallel efficiency with higher number of cores (up to 1024) and they also found comparable results on parallel efficiency as the previously cited work.

In a motivation study (see Section 6) using a simple geometry it was confirmed that the partitioning method has a considerable effect on overhead at different numbers of processor cores [17]. These initial findings were the foundation for further investigations on this issue in more complex geometries.

2. Methodology of this study

In this work the effect of different partitioning methods on computational time and also parallel overhead for different methods with different numbers of processor cores was investigated on two different hardware architectures. Of course the authors are aware that the results of such a study depend on the geometry and the CFD models used for the investigation. Even if there are standard tests for parallel performance it was decided to use two very practical examples similar to common industrial chemical engineering projects. One case is dealing with multiphase fluid flow in a simple geometry; the second simulated geometry consists of a packed bed of adsorbent grains with a multicomponent gas flow passing through. This is a rather complex geometry containing separate solid and fluid regions.

2.1. Geometry

For the motivation study a geometry from a tutorial and a solver originally provided by OpenFOAM® were used [18]. The geometry is a closed cube filled with water and air. At the beginning there is an air bubble with high temperature and pressure in the middle of water. The expansion and rise of the bubble during time are simulated.

The geometry used for the second part of study was a packed bed of spherical adsorbents, which provides a solid–gas interaction environment with active surface for adsorption. The geometry was created on a fully hexahedral background mesh by marking and splitting the cells into fluid and solid regions based on a STL file describing the positioning and size of the spheres in the bed. Since the mesh was a perfect hexahedral mesh it helped to remove the effect of variation of mesh quality on different partitions from the study. The created geometry had two regions (solid and fluid) with complex structure of each region because of the randomness of the packing.

2.2. Solver

Two different solvers were used for the two parts of the study to cover a wide range of flows ranging from pure non-reactive multi-phase flow to a more complex solver which counts for all phenomena which are relevant for chemical engineering flows including mass and heat transfer. In the motivation study an original solver from OpenFOAM® package was used which solves for multi-phase flow (compressibleInterFoam). For the second part of this investigation a new solver, based on available infrastructure provided by OpenFOAM®, was used. The new solver was specially designed for dealing with adsorption phenomena [19], in which heat and mass transfer happens between gas and solid in multi-regions (fluid and solid). The solver treats each of the two regions with individual approaches; in the fluid region Navier–Stokes equations are solved coupled with continuity equation to calculate implicitly connected pressure and velocity field of the fluid. After that heat and mass conservation equations are solved explicitly for calculation of temperature and species mass fractions in the fluid. On the other hand on the solid side the energy conservation equation is solved to resolve the temperature field inside the solid particles. Adsorption is considered as a surface phenomenon so no mass conservation equation is solved for the solid region. Then the two regions are explicitly coupled through shared boundary condition between both of the regions (adsorption active surface).

2.3. Investigated parameters

In this research mainly following parameters were investigated:

Execution Time (ET): Clock time needed in reality for finishing the simulation (in some references also named wall clock time or calculation time).

Total CPU Time (TCT): Accumulated overall CPU time needed for the simulation, calculated by multiplying the execution time by number of processor nodes used (assuming equal 100% load of all involved CPU cores).

$$TCT = ET \times number\ of\ processor\ cores. \tag{1}$$

Speedup: A very well-known term in comparing parallel efficiencies, which shows the relation between number of processor cores used and the increase in calculation speed gained. Speedup indicates the decrease in ET obtained by increasing the number of processing cores relative to a base case for comparison. The base case uses the same mesh and settings except for the number of processor cores [20].

$$Speedup = \frac{ET_{Base\ case} \times number\ of\ processor\ cores\ used}{ET_{Current\ case}}. \tag{2}$$

Initial Processing Time (IPT): ET needed to load and spread the case between processor cores. IPT becomes important in the cases where simulation time is rather small or frequently restarts are necessary, e.g. if the maximum runtime for a queuing system on the server is low. IPT is a combination of the time needed for loading the case and spreading the case between processor cores. Communication including loading of cases usually is carried out by a single master process. Therefore it is assumed to take a constant time for all variants. Time for spreading the case can vary significantly even if an identical mesh is partitioned in different ways: For each method the memory addresses for the individual cell data in each processor core are not the same, data transfers will therefore result in different initial processing times.

Energy consumption: The amount of energy needed for a simulation to be finished, which is directly connected to operational costs for a simulation. Rather than plain simulation execution time an Overhead Factor (OF, other parts of energy consumption like power distribution, cooling and efficiency of components) also plays an important role in the energy consumption [20].

$$Energy\ consumption = \frac{CPU\ power}{Number\ of\ cores\ per\ node} \times OF \times TCT. \tag{3}$$

Cell count per processor core: The calculation of the number of cells per processor core needs some consideration. Since this case contains multiple regions which are computed sequentially on each of the parallel cores, the actual number of cells per core has to be adapted to avoid distortion of the results. The two regions in this case have approximately equal numbers of cells, so an average number of cells, per region was used for calculations. Therefore the number of cells per core is simply calculated by dividing the average number of cells per region by number of processor cores.

Some other aspects have been discussed in references but have not been investigated here:

- Process pinning (significant effects have been reported) [21,22].
- Compiler optimization (e.g. 30% up to 50% difference between gcc vs. Intel compilers).
- InfiniBand® settings (optimized by VSC 3 team).
- InfiniBand® vs. Ethernet (100% InfiniBand® usage assumed in this paper).

3. OpenFOAM®

Computational Fluid Dynamics or CFD is the analysis of systems involving fluid flow, heat transfer and associated phenomena such as chemical reactions by means of computer-based simulation. One of the main reasons why CFD needs special attention is the tremendous complexity of the underlying phenomena, which precludes a description of fluid flows that is at the same time economical and sufficiently complete. Another reason is the high degree of coupling of the describing equations, leading to a large ratio of execution time per simulation time [23].

OpenFOAM® is a free, open source CFD toolbox with a steadily increasing number of users and developers from the circles of research and engineering, and is already being used by many companies. The software was published under open source license (GPL [10]) in 2004 and since then it has enjoyed growing popularity. OpenFOAM® uses fully object oriented code written in C++. The code has different levels, in the low-levels of the code base classes (e.g. fvMatrix, fvc, fvm, etc.) are implemented and in the top-level code solvers are based on these base classes. There are many standard solvers included for a wide range of continuum mechanics problems, for example, fluid dynamics, multiphase flow, combustion and heat conduction as well as structural mechanics. Since the code is fully object oriented it is fairly easy to add other solvers, which are not available within the scope of the standard library. The free software does not include any Graphical User Interface (GUI), for setup and preparation (pre-processing) the command line or a text editor can be used. This has some influence on the acceptance of the software in mainstream engineering, but does not prohibit the intensive use in education and research. The computing grid (mesh) used can also be created with an editor from the command line [24].

Other strengths of OpenFOAM® are its MPI parallelism and the availability of different partitioning methods (in OpenFOAM® also known as decomposition methods [25]). Parallel communications are wrapped in Pstream library to isolate communication details [26]. Parallelization is a part of low-level OpenFOAM® implementation, which does not need to be modified or touched during the development of top-level solvers and libraries. In theory any case can be parallelized using any number of processor cores (provided it is lower than the number of grid cells) and any available partitioning method. But in reality, the computational overhead will increase after a certain number of cells per processor core in a way that simulation will not be any faster—eventually it will become even slower. This is because of the time and resources needed for communication between the cores and nodes. In general, this applies for all partitioning methods, but maybe in a different degree. Each partitioning method provides unique individual partition structure, mesh numbering, surfaces and consequently different data transfer requirements between processor cores so the parallel simulation time will differ with each of these methods.

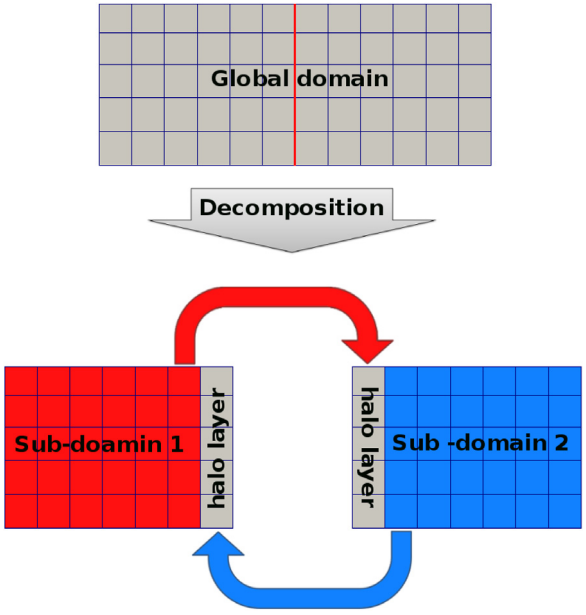

Fig. 1. Parallel implementation based on halo layer approach [30].

OpenFOAM® uses zero-halo layer parallelization strategy, in this strategy after partitioning the domain, processor boundaries are considered as internal edges and treated as boundary conditions. As it can be seen in Fig. 1 an extra layer of cells (in a n-halo layer parallelization strategy $n + 1$ layers are added to the sub-domain boundaries) is placed adjacent to these processor boundaries to mimic cells from adjacent sub-domains. In the halo layer strategy processor boundaries get explicitly updated through parallel communication calls and they are treated as implicit boundaries in solution of each sub-domain. For this approach the MPI based data communication rate will be proportional to the total area of processor interfaces. As it is expected that the number of halo layer cells is small compared to the total number of cells in the mesh the communication volume between the halo layers should be small compared to the total memory band width used. However, since latency and transfer rate between the partitions are typically slower than within the memory it is expected to see saturation effect for larger number of partitions. It is partially the intention of this study to identify this limit for common HPC systems by evaluating the speedup [27–29].

4. Partitioning methods

There are four built-in partitioning methods available in OpenFOAM®: *Scotch*, *Simple*, *Hierarchical* and *Manual* [31]. Details on the methods are provided in Table 1.

5. Machine architecture

Previous studies showed significant effect of machine architecture (CPU type, memory bandwidth, connections between CPUs and nodes like InfiniBand® connections, etc.) on the runtime and speedup of different CFD codes [33,34]. Therefore it was decided to test the effect of machine architecture (especially CPU type)

Table 1
Available partitioning *methods in OpenFOAM®* (see Ref. [32]).

Scotch	This is the most automatic method packed with OpenFOAM® for partitioning the case based on the minimizing the number of processor boundary faces. In this method, only the number of processors needs to be defined. An optional weighting factor is available for machines with differing processor performances [32].
Simple	After *Scotch*, *Simple* is the next semi-automatic method. In this method the number of processors and the number of splits in each coordinate axis direction are specified [12].
Hierarchical	Excluding *Manual*, *Hierarchical* needs most settings. For using this method not only number of processors and number of domains in direction of each coordinate axis are required, but also the order/sequence of directions needs to be defined (e.g. first partition split occurring in *z* axis direction then in X and then Y) [12].
Manual	In this method, cells should be manually assigned to different processors by user configuration [12]. The method provides most flexibility and high potential at the price of high configuration efforts. Thus, it will be not included in this investigation since it is not likely being used in engineering.

on speedup of OpenFOAM® using two well-known CPU types, AMD and Intel [35,36]. Access to the cluster systems of the VSC group [37] provided this opportunity. The selected complex industrial case was investigated on the two available cluster architectures, Vienna Scientific Cluster 2 (VSC 2—AMD CPU) and Vienna Scientific Cluster 3 (VSC 3—Intel CPU) [37,38]. For the smaller motivation study a 32 core single node shared memory multiprocessor computer (caelv.zserv.tuwien.ac.at [39]) was used which has a similar architecture as VSC 2.

VSC 2 is a High Performance Computing (HPC) system installed in May 2011 in Vienna by MEGWARE Computer. It consists of 1314 nodes, each with 2 processors (AMD Opteron 6132 HE, 2.2 GHz, 8 cores) that are interconnected via QDR InfiniBand®. Some key numbers related to VSC 2 [37,40] are:

- Total number of available processor cores: 21 024.
- Maximum available memory: 42.0 TB.
- Total energy consumption: 420 kW (peak).

VSC 3 is HPC system installed in summer 2014 at the TU Wien building at the Arsenal campus in Vienna by ClusterVision. It consists of 2020 nodes; each equipped with 2 processors (Intel Xeon E5-2650v2, 2.6 GHz, 8 cores/Ivy Bridge-EP family). The nodes are connected with an Intel QDR-80 dual-link high-speed InfiniBand® fabric. Energy-efficient cooling is provided by the mineral-oil based CarnotJet System of Green Revolution Cooling [41]. Some key facts related to VSC 3 [37] are:

- Total number of available processor cores: 32320.
- Maximum available memory: 123 TB.
- Ranked 85/111/137 in top 500 in the November 2014/June 2015/November 2015 in the High Performance Linpack benchmark [42].
- Ranked 86/148 in top green 500 in the November 2014/2015 [43,44].
- Total energy consumption: 450 kW (peak).
- Software environment: Centos 7, intel icc + intel-mpi compiler with – xAVX – O3 flags.

6. Motivation study

Before considering complex multi-region geometries, the partitioning method influence was tested on a simple geometry named depthCharge3D (included with multi-phase compressible solver, compressibleInterFoam from OpenFOAM®) [17]. The flow geometry consists only of a simple cube with approximately one million cells [45]. This geometry was partitioned using *Scotch*, *Simple* and *Hierarchical* (*xyz* and *yxz* partitioning vectors[1]). The simulation was run with up to 16 processor cores (1, 2, 4, 8 and 16 processor cores). The simulation was carried out for half a second; all results were scaled for one full second of real time. In Fig. 2 the differences in location and structure of the partitions obtained using *Scotch* and *Hierarchical* partitioning methods can be seen. As expected, *Scotch* tries to minimize the face numbers for the processor core boundaries and it results in arbitrary looking partitioned regions.

Fig. 3 shows the speedup for the case for the different partitioning methods and core counts. For each method speedup was calculated using the data from the same method at two CPU cores as base case. Up to 8 cores reasonable speedup can be found; at higher core counts some overhead can be detected. The speedup also differs for the various methods; the difference grows with increased processor core number. Another interesting parameter was IPT; distinct differences for the various partitioning procedures are visible in Fig. 4.

For the simple case the peak memory consumption (both virtual memory and resident set size) for the different partitioning methods at different number of processor cores were extracted and plotted. As expected and it can be seen from Fig. 5 at the beginning amount of memory needed per processor core decreased by increasing number of partition, this is because of the decrease in mesh size for each processor core. After a certain number of

[1] *xyz* is the order of partitioning directions, e.g. *yxz* means the geometry was first partitioned in the *y* direction then in the *x* direction and at the end in the *z* direction.

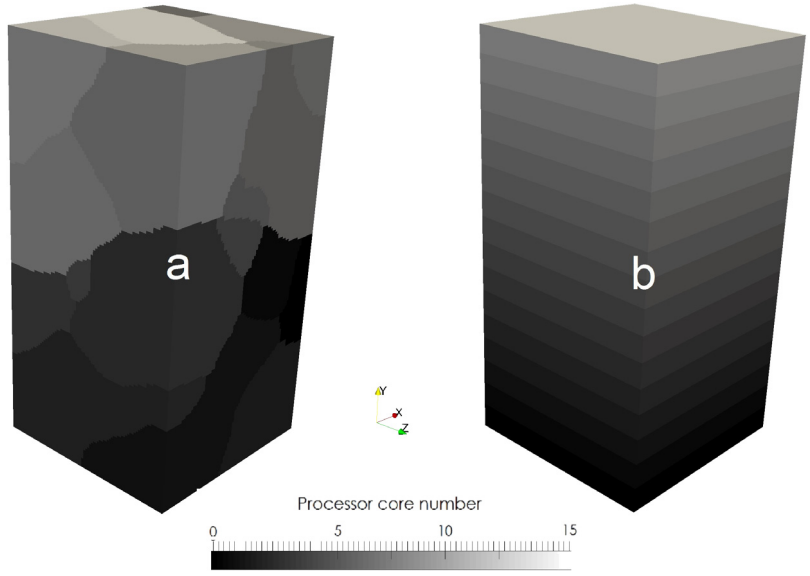

Fig. 2. Motivation study test case "depthCharge3D" partitioned using (a) Scotch (b) Hierarchical *yxz*.

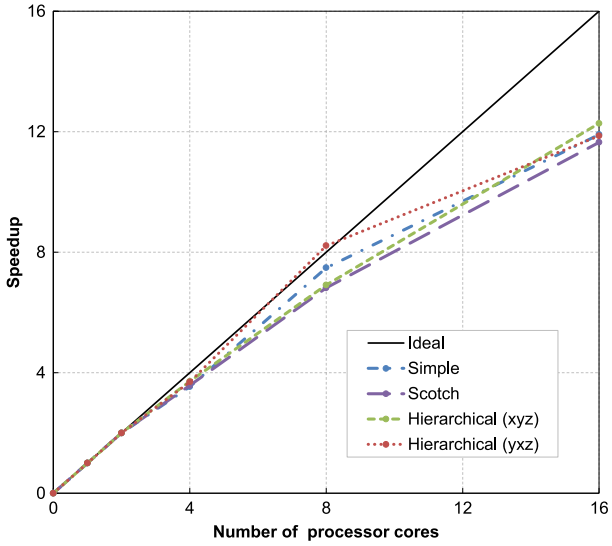

Fig. 3. Different partitioning method speedups at different number of processor cores.

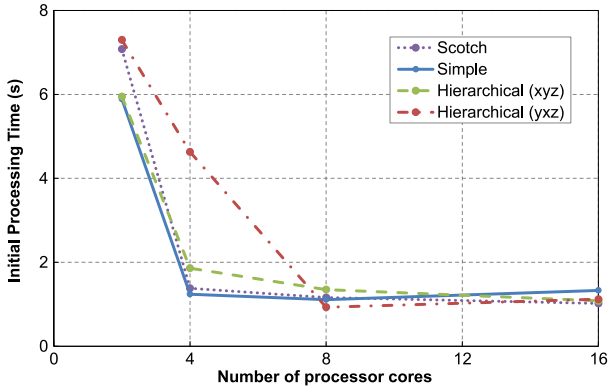

Fig. 4. IPT at different number of processor cores using various methods.

processor cores the curves level off, this can be justified with the other memory overheads e.g. the needed models and libraries

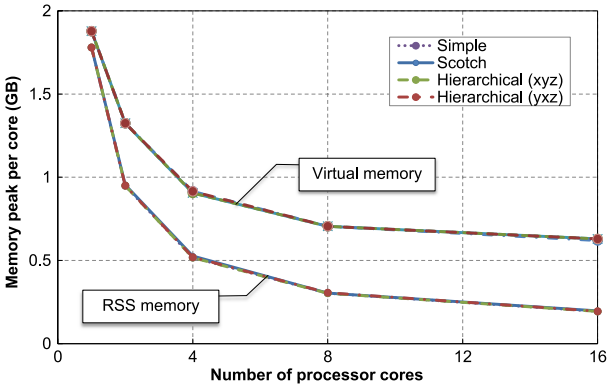

Fig. 5. Memory peak per core at different number of processor cores using various methods.

which should be loaded separately for each processor core. The other important fact which can be extracted from this data is the similar memory consumption of all for partitioning strategies at the same number of processor cores.

7. Models and algorithms

CFD relies on solving multiple conservation equations e.g. momentum, mass and energy. These equations are coupled through different models and sink and source terms. Necessary models, equations and coupling algorithms for resolving adsorption phenomena are listed below and also a solver based on a new algorithm suggested in the following was developed for this purpose.

7.1. Momentum and continuity equations

In fluid mechanics pressure and velocity are very closely coupled, this means by defining the velocity field also the pressure field will be defined and the other way round. The point is that often none of these fields are known from very beginning and they should be both calculated from boundary and initial conditions in conjugate. In CFD these two variables are calculated using two

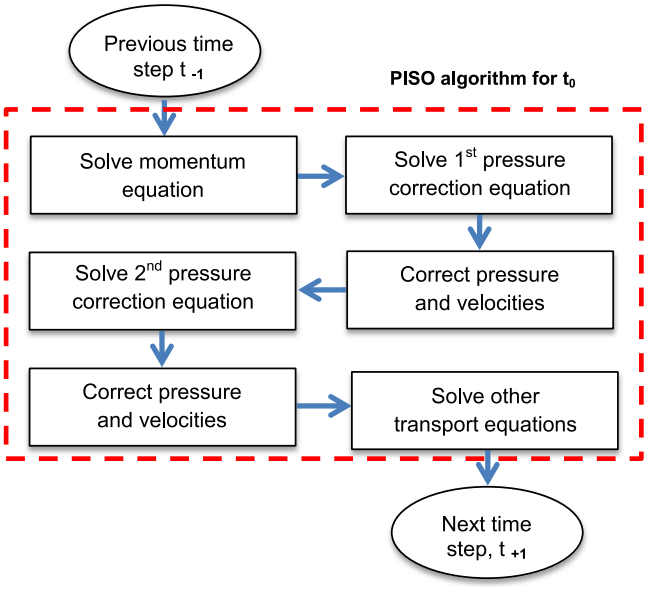

Fig. 6. PISO algorithm for solution of coupled Navier–Stokes equations.

nonlinear implicitly coupled equations, the momentum balance (Navier–Stokes) and continuity [3]:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \boldsymbol{u}) = 0 \tag{4}$$

$$\frac{\partial \boldsymbol{u}}{\partial t} + (\boldsymbol{u}.\nabla)\,\boldsymbol{u} = -\frac{1}{\rho}\nabla p + \frac{\mu}{\rho}\nabla^2 \boldsymbol{u} \tag{5}$$

where ρ [kg/m^3] is the density, u [m/s] is the velocity, p [Pa] is the pressure and μ [kg/(s.m)] is the viscosity. There are different algorithms for solving these two equations. Pressure Implicit with Splitting of Operator (PISO) algorithm is one of the well-known coupling methods for solving these two coupled equations which was originally developed for non-iterative solution of transient compressible flows [46,47].

As it can be seen in Fig. 6 first discretized momentum equations using the pressure values from previous time step are solved and based on the calculated velocities and fluxes the first pressure correction equation is solved and pressures are corrected. Based on the calculated pressures, velocities and fluxes are corrected and then the second pressure correction equation is solved. Based on the last calculations pressure and velocity fields and also boundary conditions are updated and then the time is advanced for the calculation of next time step.

7.2. Energy equation

The energy storage and transport is modeled using energy equation [3]. For fluid region the equation is as following:

$$\rho\left(\frac{\partial h}{\partial t} + \nabla\cdot(h\boldsymbol{u})\right) = -\frac{Dp}{Dt} + \nabla\,(K\nabla T) + (\bar{\tau}\nabla)\,\boldsymbol{u} + S_E. \tag{6}$$

For solid region the energy equation simplifies to following equation:

$$\rho\left(\frac{\partial h}{\partial t}\right) = \nabla\,(K\nabla T) + S_E \tag{7}$$

h [J/kg] is enthalpy, K [W/(m.K)] is thermal conductivity of the fluid and τ [Pa] is the shear stress. S_E is the energy source term which can be a volumetric source term (e.g. reaction heat) or a surface source term (e.g. adsorption heat).

7.3. Species transport equation

The conservation of chemical species i is modeled using the equation below

$$\frac{\partial \rho Y_i}{\partial t} + \nabla \cdot (\rho \boldsymbol{u} Y_i) = \nabla \cdot (D_i \nabla Y_i) + S_M \tag{8}$$

where Y_i is the mass fraction of specie i, D_i is the diffusion coefficient and S_M is the net rate of production of species i, which can be a volumetric source term (e.g. reaction) or a surface source term (e.g. adsorption). If there are n species in the system for $n-1$ species Eq. (8) will be solved and for minimizing numerical error the nth specie will be calculated using following equation:

$$Y_n = 1 - \sum_{1}^{n-1} Y_i. \tag{9}$$

7.4. Adsorption

Adsorption is a wide-spread chemical engineering unit operation referring to the enrichment of specific molecules from a fluid phase (gas or liquid) on a solid surface. Since the demand for selective separation processes will continue to rise, the need and importance of adsorption will increase. The ongoing development of adsorption processes requires very specific adsorbents and in parallel, it is essential to develop calculation and simulation methods to increase the predictability of separation processes [48]. There are two important aspects in simulation of adsorption phenomena, equilibrium and kinetics.

Adsorption equilibrium is usually described by adsorption isotherms. Isotherms are mostly empirical relations which give the adsorbate amount on the adsorbent surface or mass, depending on partial pressure (gas) or concentration (liquid) of the adsorbing component in the surrounding phase at constant temperature. For small partial pressures or concentrations a simple linear adsorption isotherm (Henry isotherm) can be used for describing the equilibrium. Under the assumption that all adsorption sites are energetically equivalent and that no interactions between the adsorbate molecules occur, the following relationship can be formulated for the Henry adsorption isotherm for a single adsorbing component [49]:

$$q_e = K_H P \tag{10}$$

where q_e [kg/m^2] is surface coverage and P [Pa] and K_H [kg/m^2/Pa] are partial pressure and Henry's adsorption constant, respectively. Other more common isotherms are e.g. Langmuir (includes saturation of the adsorbents) or BET for considering capillary condensation effects. More complex models will also be able to handle multicomponent adsorption, e.g. IAST [48,50].

The transient state until reaching equilibrium is described by the adsorption kinetics. The kinetics of an adsorption process defines the net rate of transport of molecules from the fluid to the adsorbing substances, through the pores of the adsorbing substance to the surface, the actual adsorption process on the surface and the release of the adsorption enthalpy [48,51].

From a technical and computational point of view the selection of a simple homogeneous model is preferred, since the required parameters to separate the distinct steps of the process are typically not available. Furthermore the computational effort to include a pore-wise spatially resolved multistage adsorption model for the active surface at molecular scale is currently too high for simulations in technical scale. For describing the kinetics of adsorption phenomena in this study, an empirical mixed order kinetics was

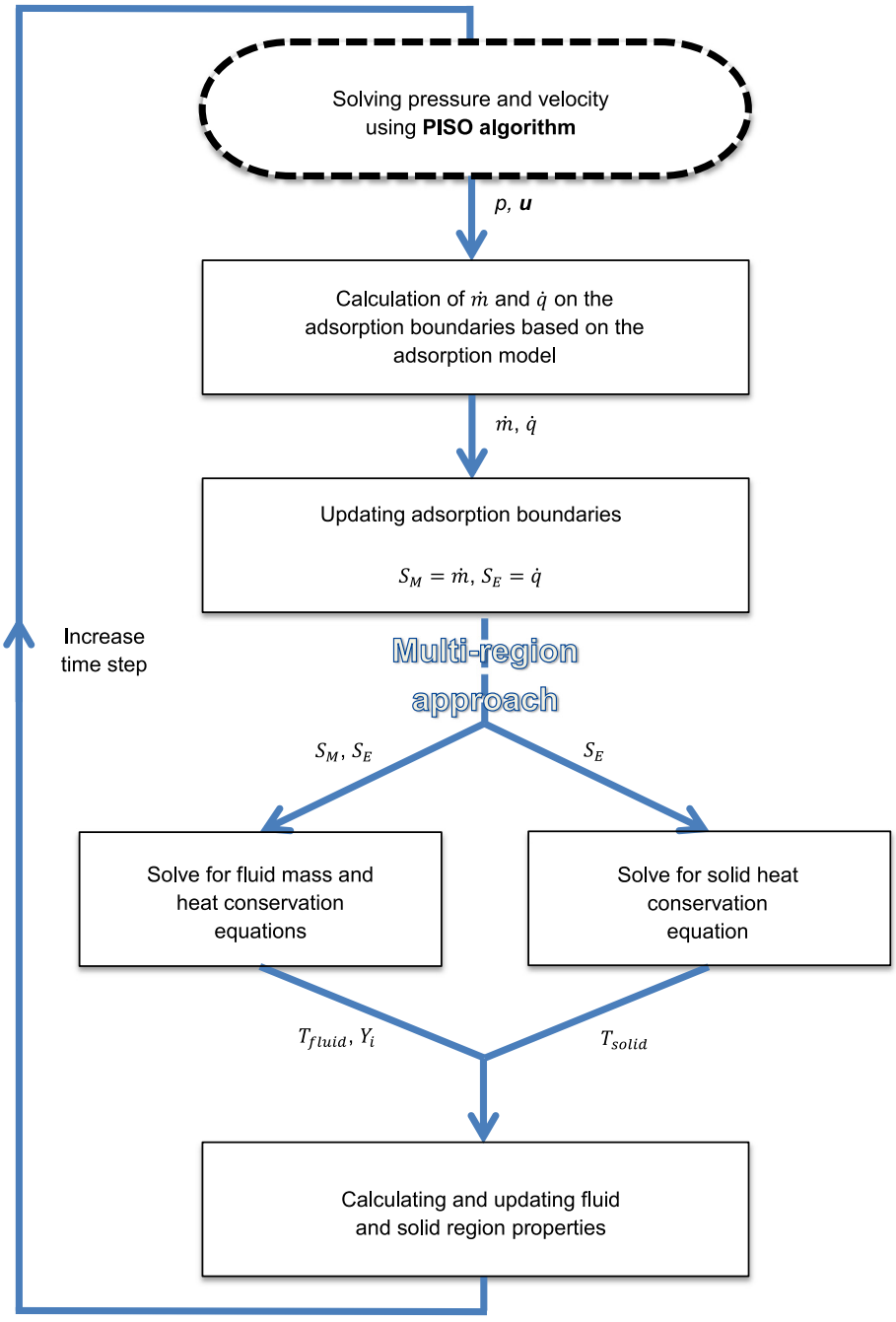

Fig. 7. adsorpFoam solver flowchart.

chosen, a combination of first and second order [52], providing a very good fit for the experimental adsorption rate data [53].

$$\dot{m} = k_1(q_e - q_t) + k_2(q_e - q_t)^2 \tag{11}$$

where k_1 [1/s] and k_2 [m^2/(kgs)] are adsorption rate constants, q_e [kg/m^2] is the equilibrium capacity which is calculated from the adsorption isotherms and m [kg/m^2] shows the amount of adsorbed component per adsorbent.

The heat released during the adsorption process can be calculated from the adsorption enthalpy

$$\dot{q} = \frac{dm}{dt} \times \Delta H_{ads} \tag{12}$$

where q [J/m^2] is the amount of released heat and ΔH_{ads} [J/kg] is the adsorption enthalpy.

7.5. adsorpFoam solver

A new solver for modeling adsorption was developed based on the multi-region approach in OpenFOAM®. Necessary extensions have been added (mass and energy source and sink terms) to model the local desorption and adsorption rates assuming a homogeneous adsorbent surface, and also taking into account the generated heat by adsorption. As it can be seen from Fig. 7 at the beginning of the time step Navier–Stokes equations are solved using PISO algorithm for calculation of pressure and velocity field. Then based on the adsorption model, adsorbed mass and heat source terms on the boundaries are calculated for each cell and time step and source terms for heat and mass equations are updated. In a multi-region approach fluid and solid regions are treated separately and they communicated with each other

Table 2
Geometry dimensions and parameters.

Bed radius (m)	0.075
Bed height (m)	0.4
Number of mesh cells	6×10^7
Mapped spheres diameter range (m)	0.01 – 0.02
Number of mapped spheres	2000
Total spheres surface (m^2)	1.83

explicitly through their shared boundaries which handle the adsorption. In the fluid region energy equation and mass fraction equations for different species are solved and fluid temperature and species mass fractions are calculated. In the solid region just heat transfer equation is solved and solid temperature is calculated. Using the calculated temperatures, pressures, velocities and mass fractions the fluid and solid properties (e.g. density, heat capacity, etc.) are updated based on the available models in the solver.

8. Packed bed adsorber geometry

The base geometry consisted of a cylindrical packed bed, which was created and meshed with the commercial software ANSYS GAMBIT® [6] using unstructured hexahedral mesh elements (Table 2). The random packing of the spheres was created using DPMFoam (a DEM solver of OpenFOAM®). The positions of the spheres were mapped into a base mesh (Fig. 8). The generated multi-region geometry provided a rather complex geometry containing arbitrary arranged solid particles with a diameter distribution in the solid region and the void space in the fluid region.

9. Packed bed partitioning

The generated multi-region (gas and solid adsorbents) geometry was partitioned with different methods. For this complex multi-region geometry *Simple* and *Hierarchical* provide a similar partitioning but with different locations of the partitions. *Scotch* demonstrates a completely different behavior. It is trying to minimize the number of boundary faces between processor cores on a per-region basis as in multi-region cases each region is treated as an independent cell zone during partitioning. The partitioning of the packed bed multi-region geometry can be seen in Fig. 9.

The different simulation configurations consisting of partitioning method, number of CPU core used and choice of the hardware infrastructure are listed in Table 3. The partitioning method shows the method used for splitting the regions. The partitioning vector presents the number of partitions used in each direction for Simple and *Hierarchical* as a combination of *x*, *y* and *z* directional splits (for hierarchical method the code *xyz* shows in which order of directions the geometry was partitioned, e.g. *zxy* means the geometry was first split in the *z* direction then in the *x* direction and at the end in the *y* direction). Finally it is indicated on which hardware the simulation was run (VSC 2 or VSC 3).

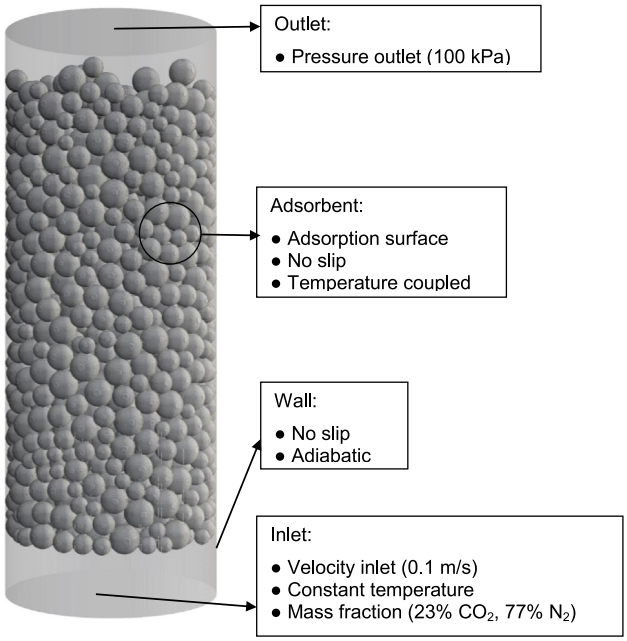

Fig. 8. Bed geometry with solid adsorbent region and process boundary conditions for CFD simulation.

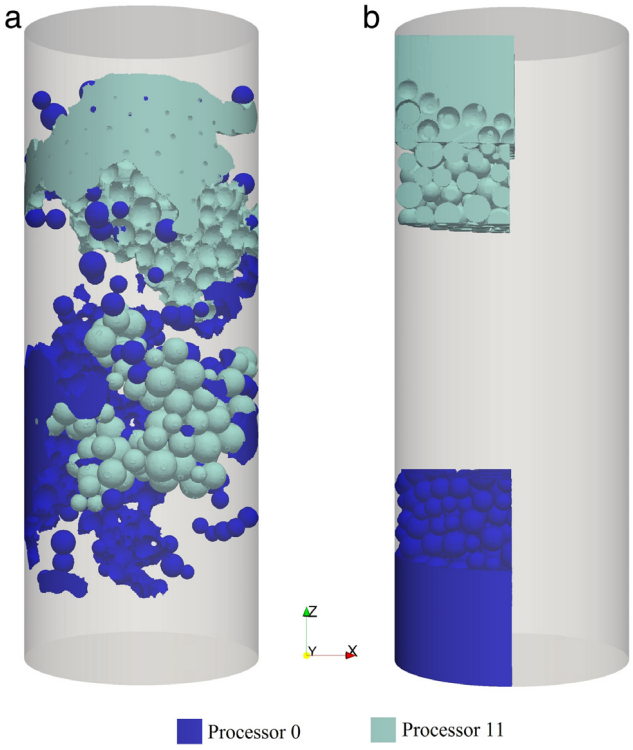

Fig. 9. Partitioned zones for a complex multi-region geometry for 16 processors (regions for processor core 0 and processor core 11 are displayed): (a) Scotch (b) Hierarchical (*zxy*).

10. Simulation settings

The whole gas domain was initialized with 100% N_2. The ideal gas equation was used to calculate the temperature dependent gas density and to represent compressibility. A gas mixture of CO_2 (23%) and N_2 (77%) was introduced into the domain from the bottom with a constant inlet velocity of 0.1 m/s. The operating

Table 3
Configuration of different simulations and machines used for executing simulation (reference case Scot16, marked with bold font).

Case	Number of cores	Partitioning method	Partitioning vector	VSC 2	VSC 3
Simp8	8	*Simple*	(2 2 2)	-	●
Scot8	8	*Scotch*	-	-	●
Simp16	16	*Simple*	(2 2 4)	●	●
Scot16 [Base case]	**16**	***Scotch***	**-**	●	●
HieraX16	16	*Hierarchical* (*xyz*)	(2 2 4)	●	●
HieraZ16	16	*Hierarchical* (*zxy*)	(2 2 4)	●	●
Simp32	32	*Simple*	(2 2 8)	-	●
Scot32	32	*Scotch*	-	-	●
HieraX32	32	*Hierarchical* (*xyz*)	(2 2 8)	-	●
HieraZ32	32	*Hierarchical* (*zxy*)	(2 2 8)	-	●
Simp64	64	*Simple*	(4 4 4)	-	●
Scot64	64	*Scotch*	-	-	●
HieraX64	64	*Hierarchical* (*xyz*)	(2 2 16)	-	●
HieraZ64	64	*Hierarchical* (*zxy*)	(2 2 16)	-	●
Simp128	128	*Simple*	(4 4 8)	-	●
Scot128	128	*Scotch*	-	-	●
HieraX128	128	*Hierarchical* (*xyz*)	(4 4 8)	-	●
HieraZ128	128	*Hierarchical* (*zxy*)	(4 4 8)	-	●
Simp256	256	*Simple*	(4 4 16)	-	●
Scot256	256	*Scotch*	-	-	●
HieraX256	256	*Hierarchical* (*xyz*)	(4 4 16)	-	●
HieraZ256	256	*Hierarchical* (*zxy*)	(4 4 16)	-	●
Simp512	512	*Simple*	(8 8 8)	-	●
Scot512	512	*Scotch*	-	-	●
HieraX512	512	*Hierarchical* (*xyz*)	(4 4 32)	-	●
HieraZ512	512	*Hierarchical* (*zxy*)	(4 4 32)	-	●
Simp1024	1024	*Simple*	(8 8 16)	●	●
Scot1024	1024	*Scotch*	-	●	●
HieraX1024	1024	*Hierarchical* (*xyz*)	(8 8 16)	●	●
HieraZ1024	1024	*Hierarchical* (*zxy*)	(8 8 16)	●	●
Simp2048	2048	*Simple*	(8 8 32)	-	●
Scot2048	2048	*Scotch*	-	-	●
HieraX2048	2048	*Hierarchical* (*xyz*)	(8 8 32)	-	●
HieraZ2048	2048	*Hierarchical* (*zxy*)	(8 8 32)	-	●

temperature and pressure were 298 K and 100 kPa respectively to ensure ideal gas behavior for all components. The cylinder walls were considered adiabatic and with a no-slip velocity boundary condition, the same boundary conditions were applied on the adsorbent surface. Based on the superficial gas velocity the Reynolds number was ∼300 therefore the flow was considered to be laminar. An explicit first order Euler scheme was used for time discretization, the Courant number was limited to 1 to ensure stable time step operation. A combination of first and second order implicit upwind and linear schemes was used for discretizing divergence and Laplacian terms in the sub-domains.

11. Procedure

For all cases the same base mesh was utilized and partitioned according to Table 3 in multiple ways. The transient simulations were run with adsorpFoam for 0.5 s simulated time on the specified computing clusters. For one case (*Scotch* with 256 cores) the full results were investigated in detail using Paraview® [54] to make sure that the CFD solution was physically correct. For all remaining cases overall integral quantifiers of key variables (e.g. adsorption loadings, rates and mass fraction of the adsorbing gas at the outlet) through whole domain after 0.5 s were compared to this detailed case for making sure that the results be identical and correct. If the resulting values deviated less than a selected small number or were within a certain bandwidth, the case was considered to be valid. Usually, the deviations were in the range of 0.001% of the absolute numbers, no case had to be eliminated for wrong calculation results. However, since the main focus of this investigation was not on the adsorption process itself no in-depth evaluation was carried out. From the valid cases statistical data on the computational solution process was collected and analyzed.

To exclude one-time effects as accidentally overloaded nodes, hangs of the file system and other possible sources of error, some randomly selected runs have been repeated on the same hardware. In all the cases the results were in good agreement regarding reproducibility.

12. Results and discussion

12.1. Detailed case solution results (CFD)

In Fig. 10 different fields at $t = 0.5$ s are presented for the selected evaluation case (*Scotch* with 256 cores on VSC 3). As expected, in the contour plot of the velocity magnitude (Fig. 10a) the highest velocities occur close to the outer walls of the geometry

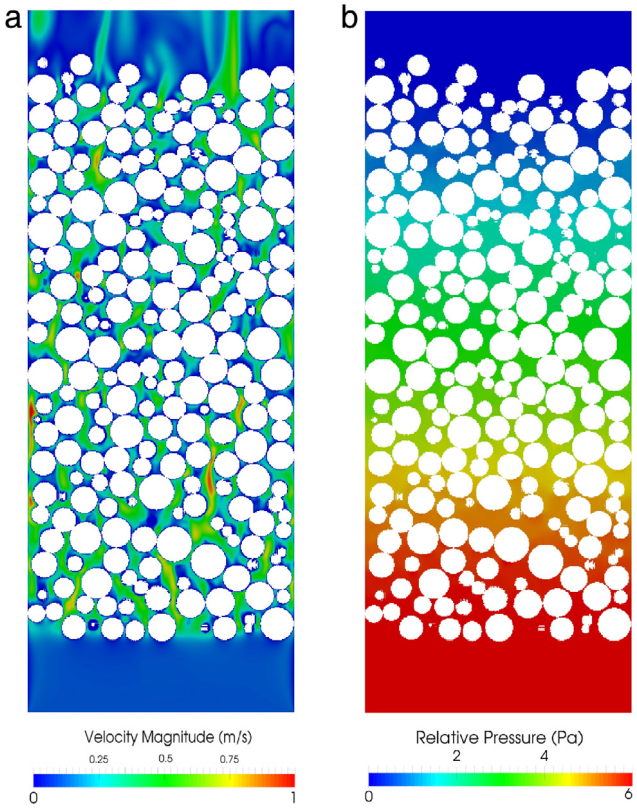

Fig. 10. Contour plots of different variables through symmetry plane of the bed at $t = 0.5$ s, (a) Velocity [m/s] (b) Pressure [Pa].

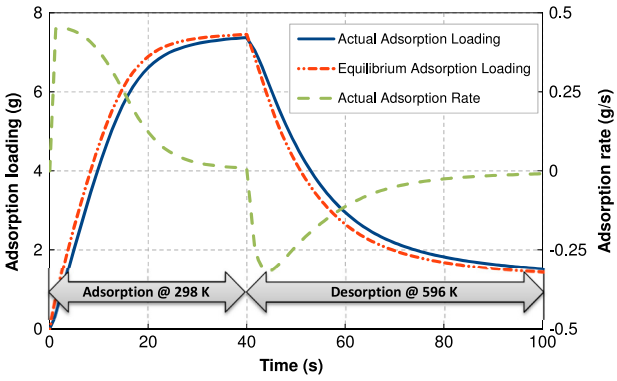

Fig. 11. Simulation of a full adsorption and desorption cycle with new solver, adsorpFoam.

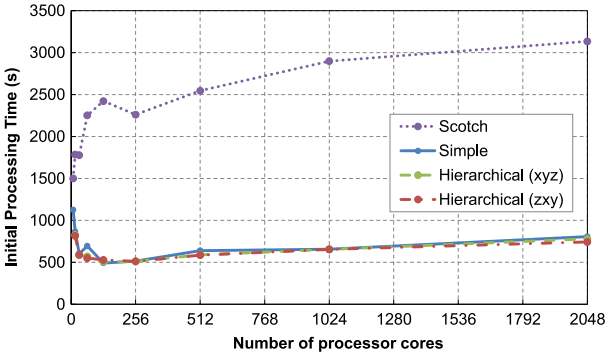

Fig. 12. Initial Processing Time (IPT) for different discretization methods at different number of processor cores.

due to irregularities and wall effects of the packing. The contour plot in (Fig. 10b) shows the static pressure field in the packing, the calculated pressure drop Δp is about 6 Pa which compares well to a prediction using the Ergun equation [50].

$$\Delta p = \frac{150 \cdot \mu \cdot (1-\varepsilon)^2 \cdot u \cdot L}{\varepsilon^3 \cdot d_p^2} = 5.7 \text{ Pa} \tag{13}$$

where, μ is the dynamic viscosity of the gas (3×10^{-5} Pa.s), u is the superficial gas velocity (0.1 m/s), ε is the void fraction of the packed bed (0.43), L is the packed bed length according to Table 2 and d_p is the particle diameter according to Table 2.

A full adsorption and desorption cycle under the same conditions explained in the simulations settings was performed. As it can be seen in Fig. 11 adsorption runs for 40 s at 298 K. At the beginning adsorption happens fast, it is because the adsorber is not saturated and the driving force is high. At the end of adsorption since the adsorbent is saturated the adsorption slows down. This behavior can also be seen from adsorption rate graph. Overall around 7.5 g CO_2 is adsorbed on the adsorber. After this stage the temperature was increased to 596 K to desorb the CO_2 and to regenerate the adsorber. Desorption needed around 60 s to remove most of the adsorbed CO_2.

12.2. Considerations regarding parallel efficiency

12.2.1. Initial processing time (IPT)

As mentioned in the motivation study this parameter changes for various methods with different number of processor cores. In Fig. 12 IPT for different methods can be seen, for all numbers of processor cores *Scotch* has the highest IPT (in some cases up to one hour). By increasing the number of processor cores in all the methods (except *Scotch*) the IPT decreases at the beginning, because by increasing the number of processors the size of mesh dedicated to each processor becomes smaller and less time is needed to load the meshes to the nodes. At higher CPU core numbers (starting from about 256) IPT starts to increase slightly because the overheads outweighs the time saved for loading a smaller mesh. For *Scotch*, IPT increases as the number of processor cores is increased, which shows the other overheads from this method are much bigger that can cover the effect of decrease in the mesh size .

As mentioned in the motivation study the memory consumption itself had no visible dependency on the partitioning methods, which means the time needed for filling memory should be similar for all the methods at the same number of processor cores. The effect of IPT becomes more important when the simulation time is rather small, by increasing the simulation time this effect will fade. Fig. 13 shows the reduced effect of IPT by increasing the simulation time from 0.1 s to 0.5 s. It can also be seen that *Hierarchical* (*zxy*) – where the first partitioning step is carried out in the coordinate axis direction with the highest number of discretized grid cells in the geometry (z direction) – has the smallest IPT/TCT ratio and *Scotch* has the biggest. From this it can be concluded that before choosing the method this factor should also be considered. Especially for simulations with short runtimes, e.g. in the case *Scotch* with 2048 processor cores, most of the time needed for the simulation is spent on the initial processing.

IPT becomes more important on HPC clusters where scheduling systems do not allow long time slots for simulations (e.g. some high memory or special computing nodes). To investigate the effect of different methods on the execution time only and to ignore the distribution time IPT will be subtracted from the execution time, since this concept seems more suitable for long lasting simulations.

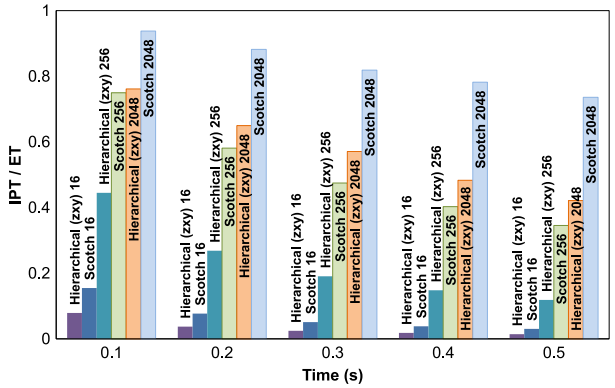

Fig. 13. Initial Processing Time (IPT) vs. Execution Time (ET) ratio.

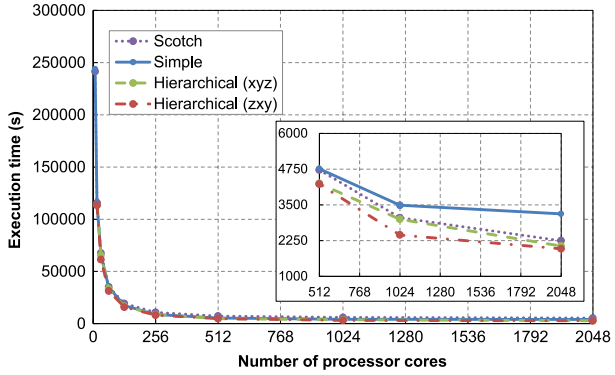

Fig. 14. Execution time (ET) per one second of simulation time excluding IPT.

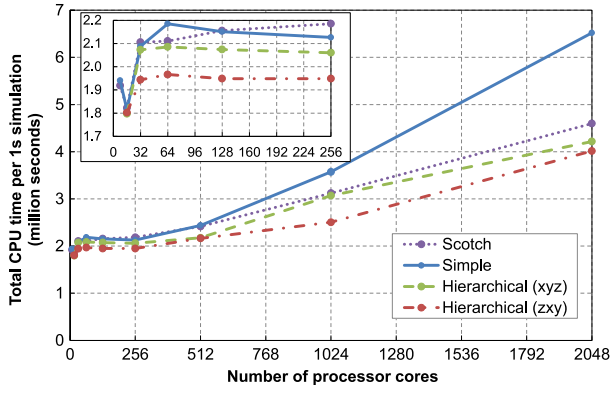

Fig. 15. Total CPU time per one second of simulation time.

12.2.2. Total CPU time and execution time

As expected it can be seen from Fig. 14 that the execution time ET excluding IPT for the computation of one second of simulation time will decrease with increasing the number of processor cores (according to 1/n). After a certain number of processor cores it approaches an almost constant value due to the overheads. This trend is clearly visible for all the partitioning methods. The subgraph in Fig. 14 provides a scaled view for the range of 512 up to 2048 processor cores. At the same number of processor cores *Simple* has the highest execution time (least performing method) and *Hierarchical* with first partitioning in *z* direction has the smallest execution time (best performing method). The simulation run using the best partitioning method at 512 processor cores (*Hierarchical zxy*) is more than 10% faster than the simulation using the least performing partitioning method (*Simple*). With the increasing number of processor cores this gap becomes even bigger, e.g. at 1024 the difference between best and worst method reaches around 30%.

In Fig. 15 total CPU time consumed for one second of simulation excluding IPT is shown. This time is estimated by multiplying execution time excluding IPT by number of processor cores used for that simulation. Considering this parameter also *Simple* is the least performant (slowest) and *Hierarchical* in *z* direction is the most performant (fastest) method. The total CPU time for all methods is increasing except for the very beginning where the number of processor cores increases from 8 to 16. The reason for this unexpected behavior can be attributed to the fact that up to 16 processor cores all the calculations are done on the same node (each node has 2 CPU with 8 cores) – after that the overhead (increase in latency and limited transfer speed) of network interconnections (InfiniBand®) is added – and the utilization of the CPUs gets more efficient. This effect has not been investigated in detail, but might be addressed by processor pinning (reservations through the queuing system are node based, therefore during the simulation some cores will idle).

12.2.3. Speedup

Speedup was calculated based on the base reference case, because this was the smallest available number of processor cores for all the discretization methods. Moreover this choice was reasonable since this is equal to one full computational node, as mentioned before this configuration was most efficient for each of the partitioning methods since it resulted in the smallest total CPU time needed for doing a calculation run. In Fig. 16 the speedup (excluding IPT) for different methods can be seen. For low processor core numbers all the data points are close to ideal line (where speedup is equal to number of cores used). For higher number of processor cores the curve slope becomes smaller which shows the continuous increase in computational overheads. At some point the speedup graph becomes almost horizontal, there is no more useful gain in computation time, all of additional CPU performance is lost in communication overheads.

Simulations with less than 25% overhead are considered as acceptable. For *Hierarchical* (*zxy*) in *z* direction speedup is acceptable up to 1024 processor cores, for *Simple* the increasing overheads lead to unacceptable speedup only after 512 processor cores. In other words, depending on the method selected the efficiency differs a lot for the same core number. It is also remarkable that methods requiring more configuration data (going towards Manual) are more efficient. However, *Scotch* is also acceptable if the geometry is complex and cannot be easily partitioned using one of the more Manual methods.

As mentioned before, if the simulation is long enough it is reasonable to ignore the IPT. To demonstrate the distortion effect of IPT on speedup in this case the same graph including IPT is shown in the inset of Fig. 16. As it can be seen it has a very serious influence on calculation times and efficiencies. For such short running test cases as this one the inclusion of IPT would faultily make *Scotch* efficiency the lowest.

12.2.4. Machine architecture comparison

It is common knowledge that machine architecture has significant effects on time needed for calculations and thus the speedup. The comparison in here includes two different machines (VSC 2 and VSC 3) with different hardware and infrastructure. The comparison was done with fastest and slowest method only. One representative case for small number of processor cores (16), same was done for a high number of processor cores (1024). As it is obvious from Fig. 17 in all the cases the newer machine (VSC 3) is approximately two times faster with regard to the total CPU time for one second of simulation time. A similar trend can be observed for IPT for both machines.

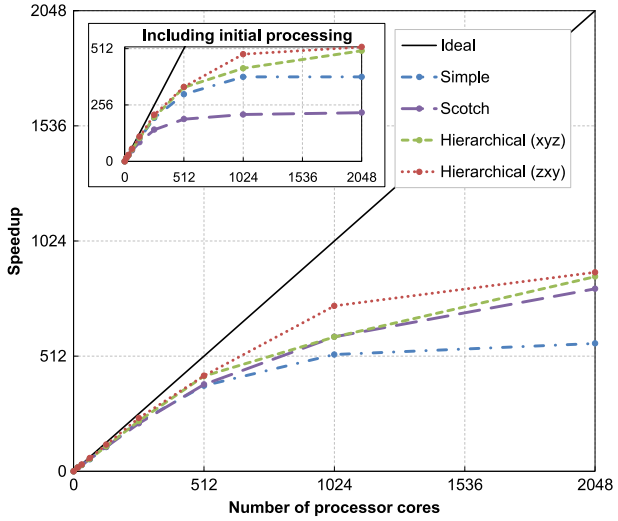

Fig. 16. Speedup of different methods at different number of processor cores.

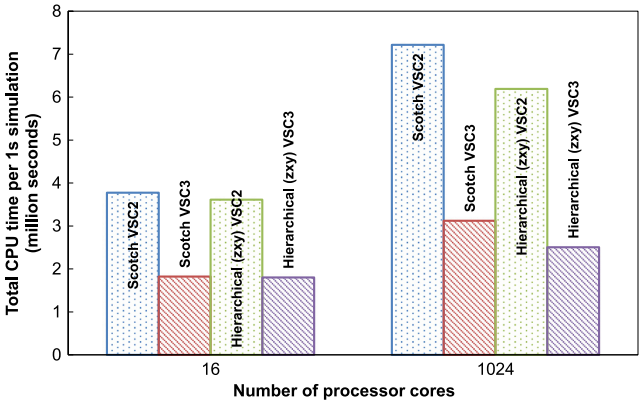

Fig. 17. Total CPU time comparison between VSC 2 and VSC 3.

12.3. Energy consumption

As discussed before, by increasing the number of processor cores the execution time for a given simulation decreases. At the first look, it seems using the maximum available number of processor cores would be a good idea. But having a closer look, it can be seen that another cost factor is also increasing which is calculation overhead. By increasing the number of processor cores not only lower execution times are obtained but also more resources are wasted. There is a tradeoff between the decrease of execution time and the increase of overall energy consumption. For a simple estimate, the energy consumption was calculated using the TDP (thermal design power) values from the data sheets of the CPU manufacturers (AMD, Intel) [55,56]—no overhead factors for other system parts (RAM, drives, network) or cooling facilities have been included since they vary for each individual system setup.

In Fig. 18 it can be seen that up to a certain critical number of processor cores (256 to 512) the energy consumption graph for VSC 3 (starting from the right hand side) is almost horizontal. This indicates that speedup within this core number range can be reached without significant additional energy consumption. For higher numbers of processor cores the energy consumption increases very fast. It was also found that the energy demand depends to a certain degree on the domain partitioning methods,

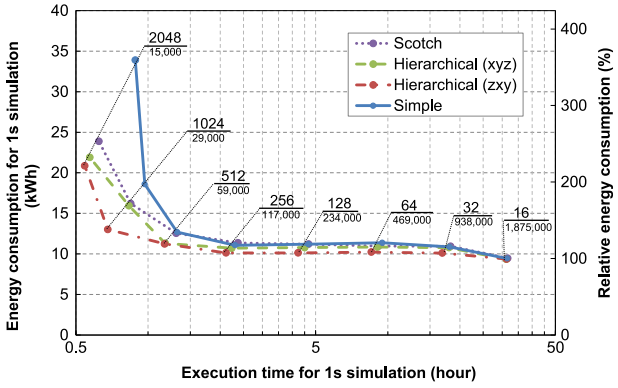

Fig. 18. Energy consumption vs. execution time at different number of processor cores on VSC 3.

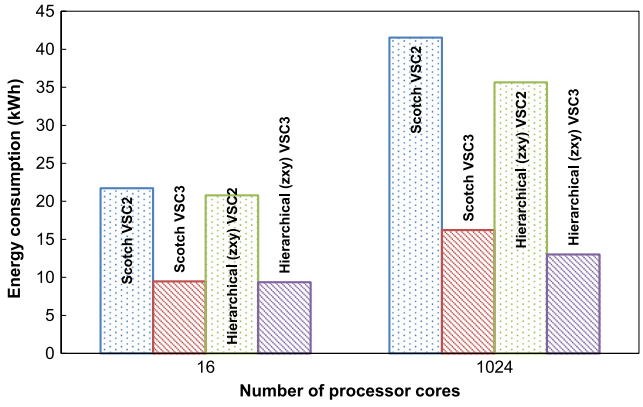

Fig. 19. Comparison of VSC 2 and VSC 3 energy consumption.

especially for higher numbers of cores. The critical number of cores needs to be determined for each architecture and CFD solver individually since the memory bandwidth of the systems and the memory utilization are always different.

Instead of the number of processor cores used for a simulation another more general parameter can be considered. This is the number of cells of the full mesh assigned to a single processor core. This parameter is also included in Fig. 18 (small print number below core count). From approximately 2 million cells per processor core down to 120,000 cells per processor core (in some methods even down to 60,000 cells per processor core) the total energy consumption is quite constant.

Comparing VSC 2 and VSC 3 from an energy consumption point of view as expected VSC 3 is much more efficient. It can be seen from Fig. 19 that VSC 3 is more than two times more efficient in energy consumption for all cases.

13. Conclusion

Multiple factors influence the parallel performance in large cases. Different partitioning methods lead to different parallel efficiency at the same number of processor cores. Generally, the more manual methods (e.g. *Hierarchical* from OpenFOAM®) are more efficient from a computational point of view. By increasing the number of processor cores the overheads also increase. But up to a certain number of cell partitions the overheads can be ignored for sake of speedup. For higher numbers of partitions efficiency decreases very fast and it is no longer efficient to use more processor cores. From an energy consumption point of view, there is a tradeoff between speed (total runtime of a simulation) and total

consumed energy. The degree of parallelization can be increased up to a certain critical number of CPU cores with reasonable energy efficiency; above the critical core count the price for little more speedup is extremely high. For the case investigated and for the considered HPC system the critical cell count was approximately 60,000 cells per core. It is evident that machine architecture also plays a very important role in speedup and efficient calculations. As observed a newer machine was more than two times more energy efficient compared to a three year older generation. One other factor to be considered especially in case of short simulations or simulations on machines with short scheduling times before selecting the partitioning method and number of cores is the initial processing time (IPT). In the case investigated here, also the most manual method (*Hierarchical*) had the best output. Unfortunately different solvers have different utilization of computing resources; therefore general recommendations can only be a rough guideline—for the best utilization of cluster systems it is advised to invest some time before doing long simulation runs on HPC systems to find the optimum partitioning methods and most advantageous number of processor cores. Green computing should not only consider energy efficient hardware, it is also required to make the most efficient use of this hardware.

Based on the findings of this work the following concluding recommendations are given:

- Speedup is solver depending but overall similar trends in speedup using different partitioning methods can be expected.
- If possible, use optimized compiler setting for the architecture.
- More manual partitions are preferred to get the most speedup out of parallelization.
- Use the principal axis of the geometry with the highest cell count along it as the main partitioning axis for the methods with direction order selection option, e.g. Hierarchical.
- Target up to 50,000–100,000 cells per processor core, this number is depending on the complexity of the solvers, by increasing the complexity the cell count should be adjusted toward the lower limit.

Acknowledgments

The computational results presented have been achieved using the Vienna Scientific Cluster (VSC, www.vsc.ac.at). Thanks to the VSC team for compiling OpenFOAM® and related packages.

Financial support was provided by the Austrian research funding association (FFG) under the scope of the COMET programme within the research project "Industrial Methods for Process Analytical Chemistry—From Measurement Technologies to Information Systems (imPACts, www.k-pac.at)" (contract # 843546).

References

[1] Suhas Patankar, Numerical Heat Transfer and Fluid Flow, CRC press, 1980.
[2] Anja R. Paschedag, CFD in der Verfahrenstechnik: Allgemeine Grundlagen und mehrphasige Anwendungen, pp. 211–220.
[3] www.computing.llnl.gov/tutorials/parallel_comp/#top. (Last accessed march 2016).
[4] www.mpi-forum.org. (Last accessed March 2016).
[5] www.openmp.org. (Last accessed February 2016).
[6] www.ansys.com/Products/Simulation+Technology/Fluid+Dynamics/Fluid+Dynamics+Products/ANSYS+Fluent. (Last accessed March 2016).
[7] www.cd-adapco.com/products/star-ccm%C2%AE. (Last accessed March 2016).
[8] www.openfoam.org. (Last accessed March 2016).
[9] Joel H. Ferziger, Milovan Peric, Computational Methods for Fluid Dynamics, Springer Science & Business Media, 2012.
[10] www.gnu.org/licenses/gpl-3.0.en.html. (Last accessed May 2016).
[11] www.openfoam.org/features/parallel-computing.php. (Last accessed March 2016).
[12] Miao Wang, Yuhua Tang, Xiaowei Guo, Xiaoguang Ren, Advanced Computational Intelligence, ICACI, 2012 IEEE Fifth International Conference on, IEEE, 2012, pp. 99–104.
[13] Shannon Keough, Optimising the Parallelisation of OpenFOAM Simulations. No. DSTO-TR-2987 Defense Science and Technology Organization Fishermans Bend (Australia) Maritime DIV, 2014.
[14] Cédric Chevalier, François Pellegrini, Parallel Computing 34 (6) (2008) 318–331.
[15] Andras Horvath, Christian Jordan, Michael Harasek, Comparison of a Commercial and Open Source CFD Software Package, 2008. www.zid.tuwien.ac.at/projekte/2006/06-166-2.pdf, ISSN: 1605-4741.
[16] Michael Harasek, Andras Horvath, Christian Jordan, Steady-state RANS simulation of a swirling, non-premixed industrial methane-air burner using edcSimpleFoam, 2011. www.zid.tuwien.ac.at/fileadmin/files_zid/projekte/2011/11-166-2.pdf.
[17] Vikram Natarajan, Personal communication, internal report by, IAESTE trainee summer 2014.
[18] www.cfd.at/OpenFoam_basic_training. (Last accessed March 2016).
[19] Bahram Haddadi, Christian Jordan, Michael Harasek, Chem. Ing. Tech. 87 (8) (2015) 1040–1041. http://dx.doi.org/10.1002/cite.201550083.
[20] Seyed H. Roosta, Parallel Processing and Parallel Algorithms: Theory and Computation, Springer Science & Business Media, 2012.
[21] www.dtic.mil/get-tr-doc/pdf?AD=ADA612337. (Last accessed March 2016).
[22] pandora.nla.gov.au/tep/24592. (Last accessed February 2016).
[23] Matthias Kraume, Transportvorgänge in Der Verfahrenstechnik: Grundlagen Und Apparative Umsetzungen, Springer-Verlag, 2013.
[24] Hrvoje Jasak, Aleksandar Jemcov, Zeljko Tukovic, International Workshop on Coupled Methods in Numerical Dynamics, vol. 1000, IUC Dubrovnik, Croatia, 2007.
[25] http://cfd.direct/openfoam/user-guide/running-applications-parallel/. (Last accessed February 2017).
[26] Yuan Liu, Hybrid Parallel Computation of OpenFOAM Solver on Multi-Core Cluster Systems (Masterthesis), Stockholm: KTH, 2011.
[27] Jasak Hrvoje, Handling Parallelisation in OpenFOAM Cyprus Advanced HPC Workshop, 2012.
[28] Sang Bong Lee, 8th International OpenFOAM Workshop, Jeju, Korea, 2013.
[29] Sang Bong Lee, Int. J. Naval Archit. Ocean Eng. 8 (5) (2016) 434–441.
[30] Carolin Körner, Thomas Pohl, Ulrich Rüde, Nils Thürey, Thomas Zeiser, Numerical Solution of Partial Differential Equations on Parallel Computers, Springer, Berlin Heidelberg, 2006, pp. 439–466.
[31] www.openfoam.org/docs/user/running-applications-parallel.php. (Last accessed March 2016).
[32] cfd.direct/openfoam/user-guide/running-applications-parallel. (Last accessed March 2016).
[33] Jamshed Shamoon, Using HPC for Computational Fluid Dynamics: A Guide To High Performance Computing for CFD Engineers, Academic Press, 2015.
[34] http://www.padtinc.com/blog/the-focus/ansys-fluent-performance-comparison-amd-opteron-vs-intel-xeon. (Last accessed February 2017).
[35] Wellein Gerhard, Peter Lammers, Georg Hager, Stefan Donath, Thomas Zeiser, Parallel Computational Fluid Dynamics: Theory and Applications, Proceedings of the 2005 International Conference on Parallel Computational Fluid Dynamics, 2006, pp. 24–27.
[36] Yue Xiaoqiang Hao Zhang, Congshu Luo, Shi Shu, Chunshen Feng, International Conference on Parallel Computing in Fluid Dynamics, 2013, pp. 149–159.
[37] vsc.ac.at. (Last accessed: March 2016).
[38] Christian Jordan, Christian Maier, Benze Kiss, Zsolt Harsfalvi, Michael Harasek, Vortrag: 3rd Vienna Scientific Cluster User Workshop, Neusiedl am See; 24022014 - 25022014; in: "book of abstracts", 2014, S 21.
[39] zid.tuwien.ac.at. (Last accessed March 2016).
[40] www.top500.org/system/177280. (Last accessed May 2016).
[41] www.grcooling.com/carnotjet. (Last accessed May 2016).
[42] www.top500.org/system/178471. (Last accessed May 2016).
[43] www.green500.org/lists/green201411&green500from=1&green500to=100. (Last accessed May 2016).
[44] www.green500.org/lists/green201511&green500from=101&green500to=200. (Last accessed May 2016).
[45] www.openfoam.org/archive/231/download/. (Last accessed 1 April, 2016, Last accessed March 2016).
[46] Iain Edward Barton, Internat. J. Numer. Methods Fluids 26 (4) (1998) 459–483.
[47] Henk Kaarle Versteeg, Weeratunge Malalasekera, An Introduction To Computational Fluid Dynamics: The Finite Volume Method, Pearson Education, 2007.
[48] Dieter Bathen, Und Breitbach Marc. Adsorptionstechnik, Springer Verlag, ISBN: 3-540-41908-X, 2001.
[49] Stephen Allen, Gordon Mckay, John Francis Porter, J. Colloid Interface Sci. 280 (2) (2004) 322–333.
[50] Douglas M. Ruthven, Principles of Adsorption and Adsorption Processes, John Wiley & Sons: New York, 1984.
[51] Duong D. Do, Adsorption Analysis: Equilibria and Kinetics, Vol. 2, Imperial College Press, London, 1998; ISBN: 1-86094-130-3.

[52] Wojciech Plazinski, Wladyslaw Rudzinski, Anita Plazinska, Adv. Colloid Interface Sci. 152 (1) (2009) 2–13.
[53] Michael Martinetz, Simulation Von Adsorption Mit OpenFOAM (Masterthesis), Vienna University of Technology, Vienna, 2014.
[54] Paraview.org, 2016. (Last accessed April 2016).
[55] products.amd.com/en-ca/search/CPU/AMD-Opteron%E2%84%A2/AMD-Opteron%E2%84%A2-6100-Series-Processor/6132-HE/27#. (Last accessed March 2016).
[56] ark.intel.com/products/75269/Intel-Xeon-Processor-E5-2650-v2-20M-Cache-2_60-GHz. (Last accessed February 2016).